AIGC 5005 | MODULE 5 | 3.5% OF FINAL GRADE

Lab 05: Predict Late Deliveries

Due date posted on Blackboard

A food delivery company wants to know, at the moment an order is placed, whether it will arrive late, so that it can offer a small voucher before the customer is disappointed. You will build the scikit-learn model that makes that call: a leak-free pipeline, an honest comparison of models against a baseline, a decision threshold chosen from the company's own costs, and a saved model that another program can load. The Module 5 worked example did this for customer churn. This lab asks you to do it for a different problem.

You start with an invented table of 3,000 orders and finish with a repository that a marker can clone, run and trust. The marks are for the quality of your reasoning as much as the code: which columns you were allowed to use, how you chose the model, how you chose the threshold, and whether you looked at the test set only once.

What this lab is about

In Module 5 you saw how easily a good-looking score can be wrong: by leaking information from the test set, by trusting accuracy on a rare class, or by tuning until the test result looks nice. This lab asks you to avoid all three on a problem you have not seen before. Nothing here is a new library feature. The difficulty is in the decisions, and every decision must be written down and defended in your README and your notebook.

The scenario

QuickBite delivers restaurant meals. About one order in six arrives more than ten minutes after the time promised to the customer, and those customers complain, ask for refunds and stop ordering. The company can act on a prediction made when the order is placed. It can offer the customer a **$4 voucher** along with an apology for a likely delay. A voucher always works: an order that is flagged and turns out late costs only the $4. An order that is **not** flagged and arrives late costs **$15** in refunds and lost repeat custom. A flagged order that arrives on time still costs the $4.

So the cost of a set of decisions is $4 for every order flagged, plus $15 for every late order that was not flagged. The company currently flags nobody. Your task is to build a model and choose a threshold that lowers the total cost, and to show honestly, on data the model has never seen, by how much.

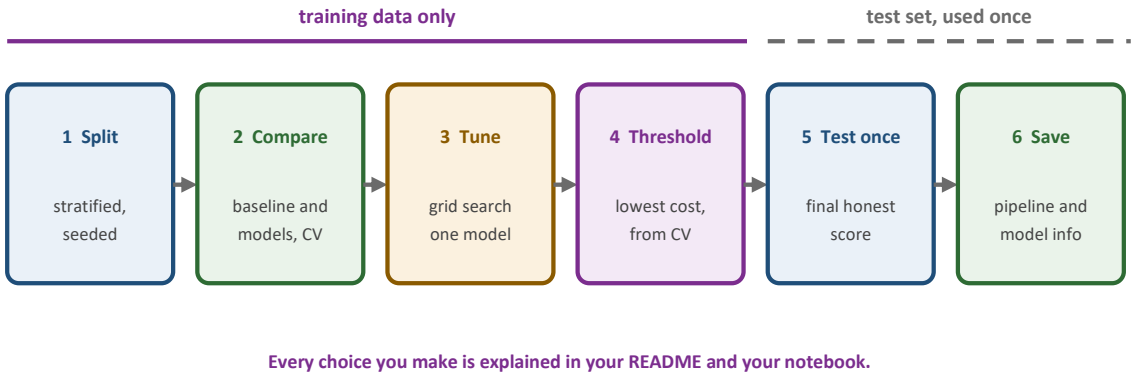


Figure 1. The path of Lab 05. Steps 1 to 4 use only the training data. The test set is scored once, at step 5.

The data

The file below, **data.py**, builds the order table from a seed, so you and every classmate get identical rows. Save it, unchanged, as src/latedeliveries/data.py in your repository. Do not edit it. It needs only NumPy and pandas, which scikit-learn already brings with it. The target column is **late**, which is 1 for an order that arrived late.

Column	Meaning
order_id	A number that labels the order
distance_km	Distance from restaurant to customer, in kilometres
items	Number of items in the order
hour	Hour of the day the order was placed, from 10 to 22
weather	clear, rain or snow
day_type	weekday or weekend
vehicle	bike, scooter or car
prep_minutes	Minutes the kitchen took to prepare the order. Sometimes missing
courier_months	How many months the courier has worked for QuickBite
customer_rating	The rating the customer gave, from 1 to 5. Many customers never give one
late	The target: 1 if the order arrived late, otherwise 0

```
"""An invented table of food delivery orders, for Lab 05."""

import numpy as np
import pandas as pd


def make_orders(n: int = 3000, seed: int = 7) -> pd.DataFrame:
    """Return n invented orders. late is 1 when an order arrived late."""
    rng = np.random.default_rng(seed)
    distance = np.round(rng.gamma(2.2, 1.8, n).clip(0.3, 15), 1)
    items = rng.integers(1, 9, n)
    hour = rng.choice(np.arange(10, 23), n)
    weather = rng.choice(["clear", "rain", "snow"], n, p=[0.72, 0.22, 0.06])
    day_type = rng.choice(["weekday", "weekend"], n, p=[0.68, 0.32])
    vehicle = rng.choice(["bike", "scooter", "car"], n, p=[0.35, 0.40, 0.25])
    prep = np.round(rng.normal(14, 5, n).clip(4, 40), 1)
    courier_months = rng.integers(0, 60, n)
    rush = ((hour >= 12) & (hour <= 13)) | ((hour >= 18) & (hour <= 20))
    bad_weather = [weather == "snow", weather == "rain"]
    weather_effect = np.select(bad_weather, [1.3, 0.7], 0)
    vehicle_effect = np.select(
        [vehicle == "bike", vehicle == "scooter"], [0.35, 0.0], -0.15
    )
    score = (
        -4.2
        + 0.38 * distance
        + 0.09 * (prep - 14)
        + 0.08 * items
        + 0.95 * rush
        + 0.60 * (day_type == "weekend")
        + weather_effect
        + vehicle_effect
        - 0.022 * courier_months
    )
    late = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    rating = np.clip(np.round(4.6 - 1.6 * late + rng.normal(0, 0.6, n)), 1, 5)
    frame = pd.DataFrame(
        {
            "order_id": np.arange(100000, 100000 + n),
            "distance_km": distance,
            "items": items,
            "hour": hour,
            "weather": weather,
            "day_type": day_type,
            "vehicle": vehicle,
            "prep_minutes": prep,
            "courier_months": courier_months,
            "customer_rating": rating,
            "late": late,
        }
    )
    # some values are not recorded
    frame.loc[rng.random(n) < 0.06, "prep_minutes"] = np.nan
    frame.loc[rng.random(n) < 0.50, "customer_rating"] = np.nan
    return frame
```

> **Watch out**
>
> Your model will be used at the moment an order is placed. A column is only a legitimate feature if its value is known at that moment. Decide which columns pass that test before you write any other code, and explain every column you leave out in your README.

What your project must contain

Each of the following is checked directly during marking.

1. A leak-free split and feature list. Use make_orders() from the provided data.py, unchanged. Define the feature columns once, in your code, as a numeric list and a category list, using only columns known when the order is placed. Make a stratified 75/25 split with a fixed random_state held in one constant. The test set must not be used for anything until step 5.
2. One pipeline. A function that takes any scikit-learn model and returns a Pipeline: a ColumnTransformer that fills missing numbers with the median and scales them, and one-hot encodes the categories with handle_unknown set to ignore, followed by the model. Every comparison, search and prediction uses this pipeline, never a model on its own. The pipeline must predict for an order with a missing prep_minutes and a weather value it has never seen.
3. An honest comparison. Compare a DummyClassifier baseline and at least three real candidates, one of which uses class_weight="balanced", with five-fold stratified cross-validation on the training data only. Report mean and standard deviation of accuracy, precision, recall, F1 and ROC-AUC in reports/cv_results.csv and in your README. Choose a model, and justify the choice using the costs in the scenario, not just the highest number.
4. Tuning. Run GridSearchCV on the model you chose, over at least two hyperparameters and at least six combinations, using pipeline parameter names such as model__C. State the scoring metric and why. Report the best settings, and say plainly whether tuning beat the untuned model by more than the fold-to-fold spread.
5. A threshold chosen from costs. Get out-of-fold predicted probabilities on the training data with cross_val_predict, and choose the threshold from 0.05 to 0.95 in steps of 0.05 that gives the lowest total cost under the scenario's rule. Write the cost calculation as a function. The test set must not influence the threshold.
6. One test-set result. Fit the final pipeline on the whole training set and score the test set once. Report the confusion matrix, precision, recall, F1 and ROC-AUC at your threshold, and the total cost of three policies: flag nobody, flag everybody, and your model. Write these to reports/metrics.json.
7. A saved model. Save the whole pipeline with joblib to models/late_orders.joblib, and write models/model_info.json with the scikit-learn version, the seed, the threshold, the feature lists, the best settings and the test results. Running python -m latedeliveries.train must recreate all of the above from nothing.
8. Tests. At least eight passing tests in tests/ that cover: the feature list excludes the columns you decided against, the split is stratified and repeatable, predicted probabilities are valid, the pipeline handles a missing value and an unseen category, your cost function and threshold function on a small hand-made example whose answer you have worked out on paper, and a saved model that predicts the same after reloading it (use tmp_path). At least one test must be parametrized.
9. A notebook. notebooks/analysis.ipynb, saved with its outputs, showing how common late orders are, the model comparison table, a chart of cost against threshold, the confusion matrix on the test set, and the cost of the three policies. It ends with a recommendation of no more than 200 words that answers four

questions: would you use this model, why this threshold, what did you exclude and why, and what is one limitation of your result.

10. Checks and README. python -m ruff format --check . and python -m ruff check . pass, with E, F, I, B and UP selected in pyproject.toml, and python -m pytest passes. The README follows the template below.

Required layout

The shape below is required. The names inside it are suggestions, except for the files whose names appear in the list above.

```
your-repository/
    README.md
    environment.yml          conda environment: name, Python version, pip
    requirements.txt         pinned pip packages
    pyproject.toml           package, pytest and ruff settings
    .gitignore
    src/
        latedeliveries/
            __init__.py
            data.py          PROVIDED  make_orders(), do not edit
            pipeline.py      features, the split, build_pipeline(model)
            evaluation.py    metrics, the cost function, the threshold choice
            train.py         main(): python -m latedeliveries.train
    tests/
        conftest.py          shared fixtures
        test_pipeline.py
        test_evaluation.py
        test_saved_model.py
    notebooks/
        analysis.ipynb       saved with outputs
    models/
        late_orders.joblib   the saved pipeline
        model_info.json
    reports/
        cv_results.csv
        metrics.json
```

Commit the models and reports folders. The marker needs to see your results without having to run the program first, and then checks that running it gives the same numbers.

Suggested order of work

This is guidance, not a requirement, but it keeps a large task manageable. Commit after each step.

1. Create the repository, the conda environment and the package folder. Add data.py exactly as provided, commit, then print the first rows and the late rate, and look at which columns have missing values.
2. Decide which columns are known when an order is placed. Write the feature lists and the split in pipeline.py, and write the tests for them first.
3. Write build_pipeline and test it on a missing value and an unseen category.
4. Write the comparison, including the baseline, and look at the results table before you choose anything.
5. Run the grid search on your chosen model.

6. Write the cost function and the threshold choice. Test both on a tiny example you can check by hand before you trust them on the real data.
7. Write train.py so that one command does everything, score the test set once, and save the model and the reports.
8. Write the notebook and the README, run the three checks, and test from a fresh clone.

> **See leakage for yourself**
>
> Once your project works, run your comparison a second time with the column you excluded added back, then put it back out again. Note the difference in ROC-AUC in your README. A score that improves sharply when you add one column is a warning, not good news.

Testing the cost rule

The cost function and the threshold function are the most important code in this lab, and the easiest to get quietly wrong, so test them on an example small enough to work out on paper. Here is the shape of such a test. The example is not your answer: create your own and check it by hand.

```
def test_cost_of_a_small_example():
    y_true = [1, 0, 0, 1, 0]
    flagged = [True, True, False, False, False]
    # two vouchers (2 x 4) and one missed late order (1 x 15)
    assert order_cost(y_true, flagged) == 2 * 4 + 1 * 15
```

Write at least one test where flagging nobody is the cheapest policy and one where flagging everybody is, so that you know your threshold function can choose either extreme.

Check your work

These three commands must pass before you commit a change, and all three are run on a fresh clone when your lab is marked.

```
python -m ruff format --check .
python -m ruff check .
python -m pytest
```

ruff checks notebooks as well as .py files. If it reports a problem in analysis.ipynb, run python -m ruff format . and python -m ruff check --fix . to tidy it. Settings for the tools live in pyproject.toml, so that you and the marker get identical results. Add these sections to the file you create for the package.

```
[tool.pytest.ini_options]
testpaths = ["tests"]

[tool.ruff]
line-length = 88
src = ["src", "tests"]

[tool.ruff.lint]
select = ["E", "F", "I", "B", "UP"]
```

Environment files

Create a new conda environment for this repository, and use the conda-forge channel for the Python install. The notebook needs ipykernel and matplotlib, so both belong in requirements.txt.

environment.yml

```
name: lab05-late-deliveries
channels:
  - conda-forge
dependencies:
  - python=3.11
  - pip
```

requirements.txt

```
conda env create -f environment.yml
conda activate lab05-late-deliveries
pip install scikit-learn pandas matplotlib pytest ruff ipykernel
pip freeze > requirements.txt
pip install -e .
```

> **Watch out**
>
> Run pip freeze before pip install -e . If you freeze after the editable install, requirements.txt gains a line starting with -e and an absolute path from your own computer, which will fail on every other machine. If you already have that line, delete it.
>
> Do not submit the output of conda env export. It lists every package for your operating system and will not rebuild on a classmate's machine.
>
> A joblib file can run code when it is loaded. Only load files that you made yourself or trust, and note the scikit-learn version beside the model, as model_info.json does.

Repository requirements

Create a new repository for this lab. Do not add the work to an earlier lab's repository.

Item	Requirement
.gitignore	Committed before your first code commit, excluding __pycache__/, *.egg-info/, .env, .pytest_cache/, .ruff_cache/ and .ipynb_checkpoints/. Do not ignore models/ or reports/ in this lab; the marker needs to see your output
README.md	Follows the template below, including the decisions about columns, model, threshold and the test result
environment.yml	Conda environment as described above, using conda-forge

Item	Requirement
requirements.txt	Pinned versions, including scikit-learn, pandas, matplotlib, pytest, ruff and ipykernel, with no absolute paths
pyproject.toml	Package name, version and dependencies, with src layout, plus the pytest and ruff sections
src/	The package with data.py unchanged, pipeline.py, evaluation.py and train.py
tests/	At least eight passing tests, with fixtures in conftest.py
notebooks/	analysis.ipynb, saved with its outputs
models/ and reports/	The saved pipeline, model_info.json, cv_results.csv and metrics.json from a successful run
Commit history	At least six commits, each a coherent change with a message that says what changed. data.py is added in its own commit, unchanged

README template

Replace the content, not the shape. The Decisions and Result sections carry the most marks in this lab, because they show how you reasoned.

```
# Late Deliveries

One or two sentences on the problem and what the model predicts.

## Setup
conda env create -f environment.yml
conda activate lab05-late-deliveries
pip install -r requirements.txt
pip install -e .

## Run
python -m latedeliveries.train

## Check it
python -m ruff format --check .
python -m ruff check .
python -m pytest

## Decisions
Columns used and columns left out, with the reason for each one
left out. How you chose the model, and the comparison table with
mean and standard deviation. What tuning changed. How you chose
the threshold, and why the cost rule needs it.

## Result
The test-set confusion matrix and metrics, and the cost of flagging
nobody, flagging everybody and using your model. One sentence on
how far you would trust the difference.

## What the tests cover
The behaviour your tests protect, and one thing you chose not to test
and why.

## Known limitations
Anything that does not work, or that you would improve with more time.
```

How to submit

Post the URL of your GitHub repository to the Lab 05 submission on Blackboard. Do not upload files, and do not submit a link to a single file within the repository.

```
https://github.com/your-username/your-repository
```

Make sure the repository is public. If you prefer to keep it private, you must share it with the instructor account below. Sharing with any other account does not count as a submission and will be rejected.

> **Instructor GitHub account**
>
> hossein.pourmodheji@humber.ca
>
> Add this account as a collaborator under Settings, Collaborators in your repository. Any other account will be rejected.

A repository that a marker cannot open is treated as not submitted, and the late policy applies from the point at which access is granted.

Late work

Late submissions are accepted with a deduction of 5% per day, to a maximum of five days. After five days the submission receives a grade of zero.

> **Watch out**
>
> Before you submit, clone your repository into a different folder. Create the environment from environment.yml, install requirements.txt and the package, then run the three checks and python -m latedeliveries.train. Confirm that the numbers in reports/metrics.json match the ones you committed. A repository that only works in your original folder has not passed.

> **Looking ahead to Module 06**
>
> Module 06 puts a model behind an interface that someone who does not write Python can use. The saved pipeline and model_info.json from this lab are exactly what that interface loads, and the threshold is the setting that decides what the user is told. Keep this repository tidy, because you will start from it.